

PESQUISA DE SEGURANÇA EM SISTEMAS DE VOTAÇÃO DIGITAL (LARAVEL)

1. Introdução

Sistemas de votação digital exigem um nível alto de segurança, pois lidam com dados sensíveis e decisões coletivas. Diferente de um sistema comum, aqui não basta apenas funcionar, ele precisa ser confiável, impedir fraudes e garantir o sigilo do voto.

No desenvolvimento do nosso sistema utilizando Laravel, a segurança será aplicada desde o início, tanto com recursos nativos do framework quanto com regras definidas pela própria lógica do sistema.

Além disso, é importante destacar que essas medidas podem sofrer alterações ao longo do desenvolvimento, conforme novas necessidades ou problemas forem identificados.

2. Principais mecanismos de segurança

2.1 Autenticação de usuários

A primeira camada de segurança é garantir que apenas usuários autorizados entrem no sistema.

No Laravel, isso pode ser feito com sistemas prontos como o Breeze.

Exemplo de criação de usuário com senha segura:

```
use Illuminate\Support\Facades\Hash;

$user = new User();
$user->siape = $request->siape;
$user->password = Hash::make($request->password);
$user->save();
```

2.2 Criptografia de dados

As senhas nunca devem ser armazenadas em texto puro. O Laravel resolve isso usando **hash (bcrypt)** automaticamente.

Na prática:

- Mesmo que alguém acesse o banco, não consegue ver a senha real
- Só é possível validar, nunca recuperar

2.3 Prevenção de voto duplicado

Um dos pontos mais críticos do sistema.

A solução simples é adicionar um campo no usuário indicando se ele já votou:

```
// migration
$table->boolean('votou')->default(false);
```

Na hora de votar:

```
if (auth()->user()->votou) {
    return "Você já votou!";
}
```

Após o voto:

```
$user = auth()->user();
$user->votou = true;
$user->save();
```

2.4 Sigilo do voto

Aqui está um dos pontos mais importantes.

O sistema deve:

- Saber quem já votou
- Mas NÃO saber em quem votou

Solução:

Criar tabela de votos sem ligação com usuário:

```
Schema::create('votos', function (Blueprint $table) {  
    $table->id();  
    $table->foreignId('candidato_id');  
    $table->timestamps();  
});
```

2.5 Validação de dados

Evita erros e ataques.

```
$request->validate([  
    'siape' => 'required',  
    'password' => 'required|min:6'  
]);
```

Isso garante:

- campos preenchidos
- dados válidos
- menos chance de erro no sistema

2.6 Controle de acesso

Nem todo mundo pode fazer tudo.

Adicionar um tipo de usuário:

```
$table->string('tipo')->default('usuario');
```

Verificação:

```
if (auth()->user()->tipo != 'admin') {  
    abort(403);  
}
```

2.7 Proteção contra ataques

O Laravel já protege automaticamente contra vários ataques, mas precisa usar certo.

Seguro:

```
User::where('siape', $request->siape)->first();
```

Inseguro:

```
DB::select("SELECT * FROM users WHERE siape = '$siape'");
```

Também usar CSRF nos formulários:

```
<form method="POST">  
    @csrf
```

2.8 Controle de sessão

Para evitar que alguém use a conta aberta de outra pessoa:

Arquivo:

config/session.php

'lifetime' => 30

3. Segurança durante o desenvolvimento

Durante o desenvolvimento do sistema em Laravel, a segurança será aplicada de forma contínua, seguindo estes princípios:

- Utilizar sempre recursos nativos do framework (mais seguros)
- Evitar código manual que possa gerar vulnerabilidades
- Testar validações e restrições
- Separar bem os dados (ex: voto ≠ usuário)
- Atualizar o sistema conforme novas necessidades surgirem

A segurança não é algo fixo, ela evolui junto com o sistema.